

Pipeline reproducible para integrar y analizar datos de contratación pública

TED · Open Data Portal de la Plaza (PLACSP) · DIR3 · CPV

Índice

1. Introducción	2
1.1. Pregunta de investigación	2
1.2. Contribuciones	2
2. Objetivos y alcance	3
2.1. Núcleo (entregable mínimo)	3
2.2. Extensiones (si hay tiempo)	3
2.3. Criterios de éxito	3
3. Fuentes de datos	3
3.1. Panorama	3
3.2. Volumen del corpus	4
3.3. El problema de la identidad	5
4. Arquitectura del pipeline	5
4.1. Del prototipo a la cadena final	5
4.2. Decisiones de diseño	6
5. Implementación	8
5.1. Stack	8
5.2. Punto de entrada	8
5.3. Registro y verificación de procedencia	8
5.4. Validación y control de calidad	9
5.5. Decisión del motor de Silver	9
6. Evaluación	10
6.1. Criterio de aceptación de resultados	10
7. Resultados finales	11
7.1. Ejecución E2E final	11
7.2. Productos analíticos y demostración en Tableau	11
8. Discusión	12
9. Conclusiones y trabajo futuro	13
Bibliografía	14

1 Introducción

Cada año, las administraciones públicas españolas y europeas publican decenas de miles de oportunidades de negocio en portales oficiales heterogéneos. La información existe, es abierta y gratuita, pero llega fragmentada en formatos distintos (JSON, ZIP, Atom, CO-DICE/XML), con reglas de publicación y revisión propias. Para una empresa que vende tecnología, la diferencia entre encontrar una licitación a tiempo y descubrirla cuando ya ha finalizado el plazo es, puede suponer perder una oportunidad de negocio.

Este TFM aborda ese problema desde la ingeniería de datos: construir un pipeline reproducible que **integre, normalice y estructure** los avisos de licitación publicados en TED (ámbito europeo) y en el portal de datos abiertos de la Plataforma de Contratación (OpenPLACSP), dejando el dato listo para análisis y consumo.

Integrar ambas fuentes no es un mero ejercicio de formato. Cada portal usa sus propios identificadores, sus reglas de versionado y su granularidad temporal: un expediente puede aparecer como aviso previo, rectificarse varias veces, adjudicarse o anularse, y cada uno de esos hechos llega como una publicación distinta que hay que reconciliar sin perder la traza.

También hay que registrar qué periodo cubre cada descarga, si está completo y qué contenido exacto se obtuvo. Sin esa procedencia, dos ejecuciones no se pueden comparar ni auditar.

Para una empresa proveedora, el valor está en pasar de portales fragmentados a una vista accionable: qué procedimientos siguen abiertos, con qué plazo, en qué categorías CPV encajan con su oferta y qué historial tiene el comprador. El resultado del proyecto es un dashboard analítico que permite consultar esa información sin conocer el funcionamiento interno del pipeline.

1.1 Pregunta de investigación

¿Hasta qué punto es posible construir con fuentes abiertas oficiales un histórico unificado y trazable de la contratación pública, con identidad estable de los avisos ante revisiones y anulaciones, que soporte productos de datos como un feed diario de oportunidades?

1.2 Contribuciones

- Un pipeline por capas (raw → Bronze → Silver → Gold) con procedencia explícita y rechazos localizados en lugar de descartes silenciosos.
- Un modelo canónico de eventos de contratación que preserva el histórico de avisos, revisiones y *tombstones* con identidad determinista.
- Un estado vigente derivado con Spark desde el histórico canónico, con política de apertura congelada sobre el ciclo de vida oficial de PLACSP (open_opportunities en Parquet).
- Una capa analítica ligera (DuckDB) que expone vistas reproducibles y un export determinista a CSV para Tableau, sin redefinir la lógica de negocio.
- Un contrato de datos versionado entre capas, junto con un *manifest* de ejecución con conteos y checksums.
- Un marco de evaluación reproducible (mismo resultado y rendimiento) como base para decisiones de implementación: las mejoras solo se aceptan si mantienen el com-

portamiento esperado.

2 Objetivos y alcance

El alcance se divide en un núcleo obligatorio y varias extensiones, de modo que el resultado principal sea entregable dentro del plazo del máster.

2.1 Núcleo (entregable mínimo)

- Descarga explícita y conservación local de TED (API JSON), OpenPLACSP (ZIP, Atom, CODICE/XML) y foto DIR3 de unidades orgánicas.
- Parsing y normalización a Bronze Parquet con procedencia, rechazos localizados y conteos de ingesta.
- Silver canónico: `procurement_events.parquet` con histórico de avisos, revisiones y *tombstones* como eventos, construido con Polars y verificado contra la implementación inicial.
- Gold: estado vigente resuelto con Spark desde Silver canónico (`open_opportunities` en Parquet), con política abierto/accionable congelada sobre el ciclo de vida oficial y enriquecimiento CPV/DIR3; la vista legada en memoria (TenderRecord) y el Gold provisional quedaron sustituidos.
- Capa analítica DuckDB (vistas versionadas solo-lectura sobre Gold) y export determinista a CSV para el dashboard Tableau.
- Contrato de datos versionado y *manifest* de ejecución con checksums.

2.2 Extensiones (si hay tiempo)

- Orquestación programada con Airflow sobre las funciones actuales del pipeline. El empaquetado reproducible con Docker ya está implementado.
- Feed diario de oportunidades con filtros por perfil de proveedor.

2.3 Criterios de éxito

El trabajo se considera completo cuando la cadena puede ejecutarse de forma reproducible desde los datos de entrada hasta los conjuntos utilizados por Tableau, los registros rechazados quedan identificados con su causa y el histórico de revisiones puede reconstruirse desde Silver. La ejecución debe dejar conteos, checksums de las salidas deterministas y artefactos de control suficientes para comprobar el resultado.

3 Fuentes de datos

3.1 Panorama

El corpus de referencia cubre enero-junio de 2026: seis ZIP mensuales de OpenPLACSP y un JSONL de TED, con rutas, tamaños y checksums fijados en el manifest versionado. La consulta TED de ese corpus incluye términos tecnológicos, luego está sesgada al caso de estudio y no representa el histórico completo.

Fuente	Ámbito	Formato	Papel en el pipeline
TED eSender	Europeo	JSON (API)	Avisos previos y adjudicaciones de toda la UE
Open Data PLACSP	España	ZIP · Atom · CODICE/XML	Licitaciones y adjudicaciones nacionales
DIR3	España	CSV/online	Dimensión de unidades orgánicas por ámbito
CPV	UE	Reglamento 2015/2424	Taxonomía oficial de objeto del contrato

Tabla 1 Fuentes oficiales integradas y su función en la plataforma.

La dimensión de compradores se apoya en DIR3, el directorio común de unidades orgánicas: seis ámbitos con 108.261 unidades en la foto de septiembre de 2026.

Ámbito DIR3	Unidades
AGE	19.919
CCAA	18.825
EELL	26.606
Universidades	7.360
Otras instituciones	1.338
Justicia	34.213
Total	108.261

Tabla 2 DIR3 por ámbito (foto de septiembre de 2026).

La taxonomía de objeto del contrato es el vocabulario CPV 2008: 9.454 códigos en un cuerpo de 8 dígitos más un noveno dígito de control tras un guion (XXXXXXXX-Y). El dígito de control solo detecta erratas y los ceros a la izquierda son significativos: la clave canónica es el cuerpo como cadena, nunca como entero.

3.2 Volumen del corpus

El corpus de referencia reúne datos de TED y OpenPLACSP entre enero y junio de 2026. Los seis ZIP mensuales de OpenPLACSP ocupan 1.118.920.855 bytes y el fichero JSONL de TED, 105.503.327 bytes. Los tamaños y checksums individuales se conservan en `data/gold/run_manifest.json`.

Fuente	Periodo o ficheros	Tamaño	Entradas
OpenPLACSP	Enero-junio de 2026, seis ZIP	1.118.920.855 bytes	349.399
TED	Enero-junio de 2026, un JSONL	105.503.327 bytes	9.905
DIR3	Foto de septiembre de 2026, seis ámbitos	n/d	108.261
CPV 2008	Dimensión oficial versionada	n/d	9.454

Tabla 3 Volumen de las fuentes y dimensiones del corpus de referencia.

Las fuentes difieren tanto en su empaquetado como en su granularidad. El pipeline con-

serva esos originales y los transforma en representaciones Parquet comunes para que las etapas posteriores no dependan del formato de cada portal. Los conteos de las capas derivadas se presentan en Resultados.

3.3 El problema de la identidad

Un mismo expediente puede publicarse varias veces: aviso previo, rectificación, aclaración, formalización o anulación. Cada fuente usa su propio identificador y sus propias reglas de versionado. Parte central del diseño es una identidad determinista que permita plegar revisiones sin perder el histórico: en Silver nada se sobrescribe, todo es un evento con orden y causa.

Fuente	event_id	Regla
TED	ted:notice:<ND>	Un número de aviso es un evento; repetirlo no crea filas
OpenPLACSP	placsp:notice:<atom>@<updated>	Cada updated válido es una revisión distinta
OpenPLACSP	placsp:tombstone:<ref>	El borrado es una fila, no una eliminación

Tabla 4 Identidad determinista por fuente: sin UUIDs ni relojes; reprocesar el mismo payload produce el mismo event_id.

Si aparece contenido distinto para un event_id ya existente, la transformación falla e identifica la colisión. Ninguna versión sobrescribe a la otra de forma silenciosa.

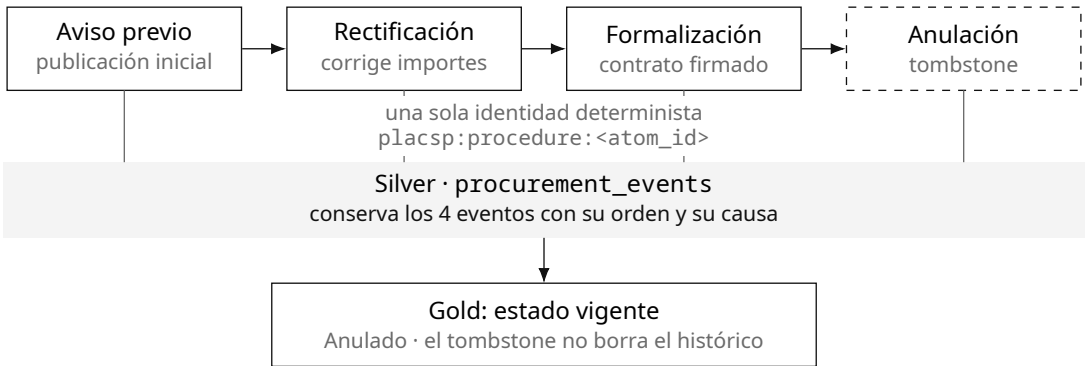


Figura 1 Plegado de revisiones: un expediente, cuatro eventos, una identidad. Silver conserva todo el histórico; Gold deriva el estado vigente sin destruir nada.

4 Arquitectura del pipeline

El diseño sigue el patrón *medallion*: capas con contrato explícito entre ellas, de menor a mayor calidad semántica.

4.1 Del prototipo a la cadena final

La primera versión completa construía Gold a partir de una vista legada en memoria que colapsaba revisiones y mezclaba lógica de negocio con ejecución. Silver pasó a ser el único contrato persistido de entrada: contiene los eventos canónicos necesarios y permite retirar esa dependencia.

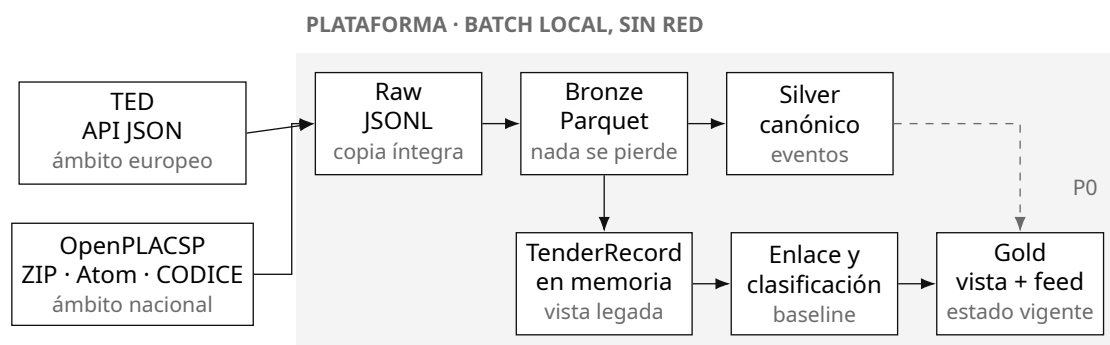


Figura 2 Vista inicial v0.1: las fuentes alimentan Raw y Bronze y Silver ya se construye como canónico, pero el Gold provisional todavía consumía la vista legada en memoria (TenderRecord) con enlace y clasificación baseline. La cadena final, con Gold derivado de Silver, se muestra en la Figura 3.

Spark obtiene el estado vigente y aplica el enriquecimiento sobre ese histórico. DuckDB queda limitado a la consulta analítica de Gold y la exportación a Tableau conserva un orden determinista. Las fronteras Parquet permiten reejecutar y auditar cada etapa sin repetir toda la cadena.

4.2 Decisiones de diseño

- Los registros que no validan van a un área de rechazos con su motivo, no desaparecen.
- Silver preserva revisiones y anulaciones como eventos; el estado vigente es una vista derivada, no el almacenamiento.
- El contrato de datos está versionado entre capas; los cambios de esquema son cambios de contrato, visibles y revisables.
- Cada ejecución emite un manifiesto con conteos y checksums de las salidas.

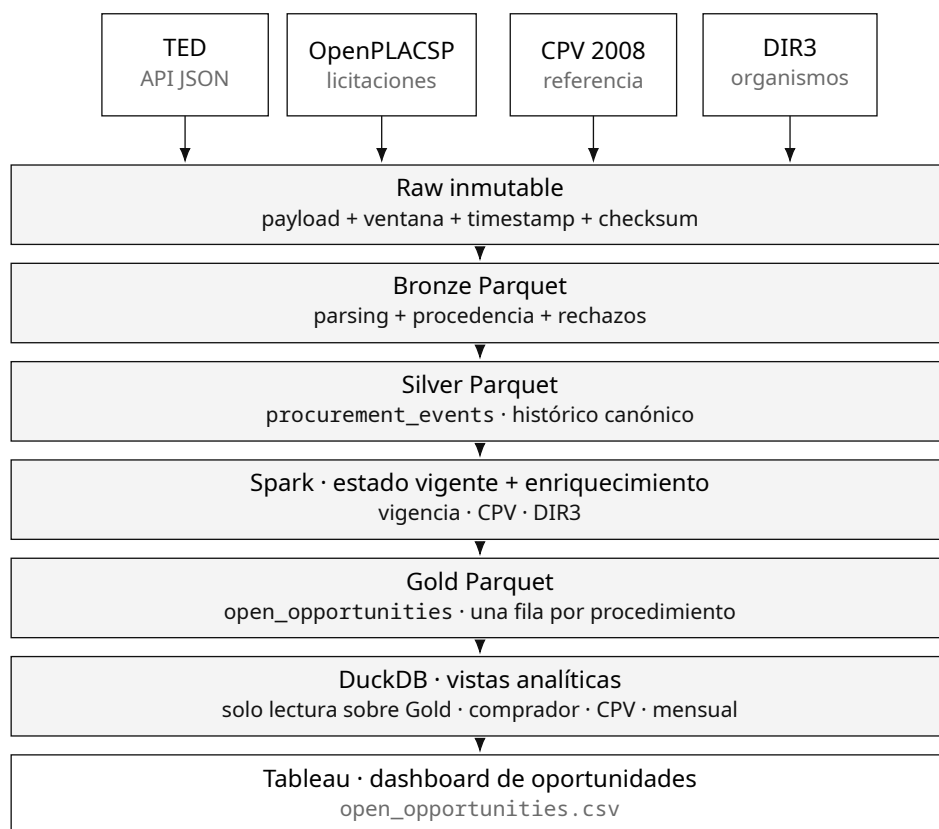


Figura 3 Arquitectura final implementada: cuatro fuentes oficiales, capas inmutables con procedencia, histórico canónico en Silver, estado vigente y enriquecimiento con Spark, Gold en Parquet, vistas analíticas con DuckDB y consumo en Tableau. La ejecución es batch local; la orquestación programada queda fuera de esta versión.

5 Implementación

5.1 Stack

Componente	Elección y motivo
Python 3.10+	Parsers de fuentes, procedencia, Silver y CLI
Polars	Procesamiento columnar de la capa Silver
Parquet	Formato columnar entre etapas
PySpark 4.0.1	Estado vigente, enriquecimiento y Gold (extra de ejecución)
DuckDB	Vistas analíticas solo-lectura sobre Gold
Tableau	Export CSV determinista + dashboard de oportunidades
uv	Gestión reproducible del entorno y de las ejecuciones
CLI(licitaciones-pipeline)	Interfaz única: ingesta, transformación, Gold, analítica y export

Tabla 5 Stack tecnológico final.

5.2 Punto de entrada

La CLI reúne los comandos de la cadena en una única interfaz. La ingesta es la única etapa que necesita acceso a red; transformación, Gold, analítica y export se pueden repetir localmente a partir de los artefactos persistidos. El siguiente listado muestra una ejecución de ejemplo, no el corte de la ejecución final.

Listing 1 Interfaz del pipeline (esquema)

```
# Ingesta explicita (es la unica etapa con red)
licitaciones-pipeline ingest --source ted \
  --start 2026-01-01 --end 2026-06-30
licitaciones-pipeline ingest --source placsp \
  --start 2026-01-01 --end 2026-06-30

# Transformacion sin red y reproducible
licitaciones-pipeline run
licitaciones-pipeline build-gold --as-of 2026-09-17
licitaciones-pipeline build-analytics
licitaciones-pipeline export-tableau
```

5.3 Registro y verificación de procedencia

Cada fichero Raw viaja con un fichero de procedencia `<fichero>.provenance.json`. Este fichero registra la fuente, la ruta, la ventana temporal, la fecha de recuperación y el checksum. Bronze comprueba esa evidencia antes de procesar el contenido y detiene la ejecución si falta o es incoherente.

El corpus histórico era anterior a la introducción de estos ficheros. Para reutilizarlo sin modificar los originales se añadió una migración de compatibilidad que comprueba el tamaño y el SHA-256 contra el manifiesto histórico y crea *hardlinks* en un directorio de trabajo nuevo. El valor `run_at` del manifiesto se registra como instante de observación del payload, no como hora original de descarga, que no estaba disponible. Esta limitación queda indicada en `legacy_migration_report.json`.

Listing 2 Fichero de procedencia (ejemplo)

```
{
  "version": 1,
  "source": "placsp",
  "raw_path": "placsp/2026-09.zip",
  "sha256": "9f2c...a41b",
  "retrieved_at": "2026-09-02T11:14:03+00:00",
  "partition": "202609",
  "window_start": "2026-09-01",
  "window_end": "2026-09-30"
}
```

5.4 Validación y control de calidad

La validación empieza en Raw, con la comprobación de la procedencia y el checksum, y continúa durante el parsing de Bronze, la construcción de la identidad canónica de Silver y los contratos de Gold. Se comprueban tipos, claves, fechas, importes, conteos entre capas y unicidad del procedimiento. Los manifiestos y los informes de ingesta conservan los conteos y el estado de estas comprobaciones.

Cada registro rechazado se escribe con un valor de `rejection_reason`, por ejemplo `invalid_json` o `source_mismatch`. El contenido original permanece en Raw y puede localizarse mediante su procedencia.

Salida Bronze	Contenido
<code>records.parquet</code>	Registros aceptados con procedencia completa
<code>tombstones.parquet</code>	Cada control de borrado válido, sin deduplicar
<code>rejections.parquet</code>	Descartados con motivo (<code>rejection_reason</code>) y alcance
<code>ingestion_report.json</code>	Conteos: <code>parsed = accepted + rejected</code> , por fuente

Tabla 6 Salidas generadas por la capa Bronze.

5.5 Decisión del motor de Silver

La implementación inicial en Python se utilizó como referencia para comparar los motores candidatos. Polars y Spark solo se consideraron válidos cuando produjeron el mismo resultado semántico que esa referencia.

Motor	Tiempo	Memoria	Resultado
Python	0,617 s	360,96 MB	Referencia
Polars	0,254 s	302,31 MB	Mismo resultado
Spark	10,010 s	1.335,80 MB	Mismo resultado

Tabla 7 Comparativa en el perfil `small` (~25.000 filas, mediana de 3 repeticiones). Informe completo en `experiments/silver_engine_comparison/`.

En el perfil `small`, Polars completó la transformación en 0,254 s frente a 0,617 s de la referencia, unas 2,4 veces más rápido. Spark mantuvo la paridad, pero sus 10,010 s no compensaron su coste de arranque en esta etapa. Silver usa Polars como ruta rápida y conserva la implementación inicial para los lotes que requieren la ruta conservadora.

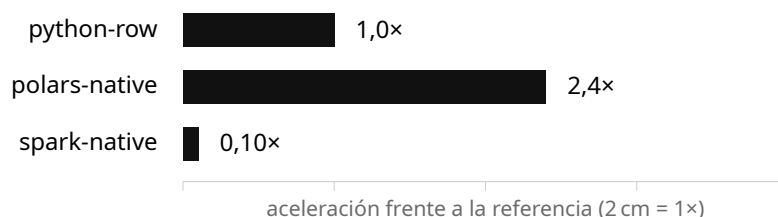


Figura 4 Aceleración frente a la referencia en el perfil `small`: Polars $\sim 2,4\times$ con el mismo resultado; Spark local queda en $0,10\times$ en esta escala por su coste de arranque. Tiempos absolutos en la tabla anterior.

6 Evaluación

La evaluación se apoya en un marco reproducible con perfiles de carga de distinto tamaño y métricas conservadas en JSON. Ninguna cifra de rendimiento se publica en la memoria sin su ejecución registrada.

6.1 Criterio de aceptación de resultados

Una implementación más rápida solo se acepta si produce exactamente los mismos resultados que la de referencia, de forma consistente en los perfiles medidos. La medición aceptó combinar Polars con la implementación inicial como respaldo; Spark no se incorporó a esta etapa del pipeline. Si los resultados difieren, la divergencia no se descarta: se documenta y se convierte en hallazgo de la memoria.

Las pruebas de integridad incluyen una colisión deliberada: si dos contenidos distintos comparten identificador, la transformación debe fallar en lugar de elegir una versión.

Perfil	TED	PLACSP	Filas	Uso
tiny	120	120	~ 310	Pruebas automáticas, milisegundos
small	10.000	10.000	~ 25.000	Comparación local
medium	100.000	100.000	~ 250.000	Escala heurística
large	800.000	800.000	~ 2 M	Exploratorio n=1 (2026-09-16)
backfill	2.000.000	2.000.000	$\sim 5,1$ M	Definido, sin resultado conservado

Tabla 8 Perfiles de prueba: conjuntos sintéticos de distinto tamaño.

Criterio	Se mide con	Umbral
Mismo resultado Bronze→Silver	Contrato semántico comparado	Igualdad exacta en todos los motores
Rendimiento de la etapa	Tiempo y memoria por perfil (JSON)	Mediana de 3 repeticiones
Cobertura de ingesta	Conteos del manifiesto frente a fuente	349.399 entradas PLACSP verificadas
Integridad de identidad	Pruebas de colisión y unicidad	Ninguna colisión resuelta en silencio

Tabla 9 Criterios de evaluación del pipeline.

7 Resultados finales

7.1 Ejecución E2E final

La cadena completa se ejecutó dos veces sobre el corpus de referencia con fecha de corte 30 de junio de 2026. Ambas ejecuciones partieron de los mismos siete payloads verificados y recorrieron Bronze, Silver, estado vigente, Gold, DuckDB y el export para Tableau. Los conteos coincidieron en todas las etapas.

Etapas	Filas
Bronze aceptadas	359.304
Rechazos Bronze	0
Tombstones Bronze	1.046
Silver canónico	359.180
Estado vigente	130.118
Oportunidades abiertas	14.763

Tabla 10 Conteos coincidentes en las dos ejecuciones E2E.

El plegado produjo 130.118 procedimientos vigentes. Gold excluyó 113.957 por estado cerrado, 521 por borrado y 877 por evidencia insuficiente. Las 14.763 filas restantes cumplen la política de oportunidad abierta para la fecha de corte. Los 9.905 eventos TED se conservaron en Silver, pero quedaron registrados como incidencias de estado vigente porque no fue posible resolver una identidad de procedimiento compatible con el contrato actual.

La reproducibilidad del producto final se comprobó comparando los cinco CSV de ambas ejecuciones. Cada par tiene el mismo tamaño y el mismo SHA-256, por lo que su contenido es idéntico byte a byte. Los manifiestos de export difieren solo en la ruta de la base DuckDB de origen, específica de cada directorio de ejecución.

7.2 Productos analíticos y demostración en Tableau

DuckDB expone vistas de solo lectura sobre Gold y el comando `export - tableau` genera cinco CSV con orden determinista. Para demostrar que estos productos pueden consumirse en una herramienta de BI se seleccionó uno de ellos, `open_opportunities.csv`, y se conectó a una visualización en Tableau. Este fichero contiene las 14.763 oportunidades, casi 15.000, que Gold considera abiertas en la fecha de corte del 30 de junio de 2026, con una fila por procedimiento. Los otros cuatro CSV se conservan como productos analíticos reproducibles, pero no forman parte de esta demostración.

CSV	Filas	Granularidad
<code>open_opportunities.csv</code>	14.763	Una fila por <code>procedure_id</code>
<code>opportunity_cpv.csv</code>	28.730	Una fila por procedimiento, CPV y posición
<code>buyer_summary.csv</code>	4.027	Una fila por identidad analítica del comprador
<code>cpv_summary.csv</code>	3.751	Una fila por <code>cpv_code</code>

Tabla 11 CSV analíticos con datos; la demostración en Tableau utiliza `open_opportunities.csv`.

8 Discusión

La identidad y el histórico condicionan los resultados. Un procedimiento puede originar varias publicaciones, por lo que contar filas sin resolver sus revisiones mezcla eventos y procedimientos. La identidad determinista y las pruebas de colisión permiten conservar esas revisiones y detectar contenidos incompatibles sin escoger una versión de forma arbitraria.

La doble ejecución aporta evidencia directa de reproducibilidad. Los conteos coincidieron desde Bronze hasta Gold y los cinco CSV finales conservaron el mismo SHA-256. Los ficheros de procedencia, los esquemas versionados y los manifiestos permiten localizar el origen de esos resultados. El perfil experimental `backfill` sigue definido, pero no se presenta como resultado porque no existe una ejecución conservada.

El benchmark local justificó Polars para Silver: en el perfil `small` redujo la transformación de 0,617 s a 0,254 s con el mismo resultado. Spark fue más lento en esa frontera, pero se utiliza en el estado vigente y el enriquecimiento, donde se aplican ventanas y joins sobre el histórico. La elección depende de la operación y del volumen medido en cada etapa.

La arquitectura final separa dos necesidades. Silver conserva todo el histórico publicado y Gold ofrece el estado vigente para análisis. Esta separación evita destruir evidencia al actualizar una vista de negocio.

Los resultados también revelan dos límites semánticos. La identidad de los 9.905 eventos TED no puede trasladarse todavía al grano por procedimiento del estado vigente. Además, ninguna de las oportunidades abiertas contiene fecha de publicación. Mantener el campo nulo evita inventar información, pero impide construir la serie mensual. Ambos problemas están en la normalización anterior a Gold y no se resuelven filtrando las oportunidades válidas.

El corpus TED usado como referencia está sesgado hacia tecnología por la consulta de adquisición. Esa limitación afecta a los resultados descriptivos del corpus, pero no restringe el modelo de datos: la ingesta, las entidades canónicas y el enriquecimiento CPV admiten contratación de cualquier sector.

9 Conclusiones y trabajo futuro

El proyecto integra fuentes heterogéneas de contratación pública en una cadena trazable. La decisión principal fue representar cada publicación como un evento con identidad determinista. Así se conservan las correcciones y anulaciones y se puede derivar el estado vigente sin perder el histórico.

Raw conserva la evidencia original, Bronze normaliza y registra rechazos, Silver mantiene el histórico canónico y Gold publica las oportunidades vigentes. DuckDB y los CSV de Tableau completan la frontera de consumo sin duplicar la lógica de negocio. Dos ejecuciones completas produjeron los mismos conteos y cinco CSV idénticos byte a byte. El resultado demuestra la reproducibilidad de la cadena para el corpus y la configuración evaluados.

El trabajo pendiente se concentra en la semántica de origen: resolver la identidad de procedimiento de TED y recuperar la fecha de publicación cuando esté disponible en la fuente. La ausencia de fecha se mantiene como nulo y no se utiliza para descartar oportunidades abiertas.

Área	Hoy	Corrección prevista
Corpus	Ventana ene-jun 2026; TED sesgado a tecnología	Ampliar histórico y filtros sectoriales
Identidad TED	9.905 eventos sin identidad de procedimiento resoluble	Revisar la extracción y el puente de identidad
Fechas	14.763 oportunidades sin <code>publication_date</code>	Recuperar la fecha upstream cuando exista
Ejecución	Batch local reproducible y empaquetado con Docker; sin orquestación	Programación con Airflow
Dashboard	Conectado solo a <code>open_opportunities.csv</code>	Añadir otros conjuntos solo si el caso de uso lo exige

Tabla 12 Limitaciones vigentes y líneas futuras: ingesta periódica, enriquecimiento con nuevas fuentes (p. ej. contratos menores) y semántica para ranking y búsqueda.

Bibliografía

- [1] Publications Office of the European Union. *TED eSender: API documentation*. Oficina de Publicaciones de la UE.
- [2] Ministerio de Economía, Comercio y Empresa. *Open Data Portal de la Plataforma de Contratación del Sector Público*.
- [3] Ministerio de Políticas Territoriales. *DIR3: Directorio Común de unidades orgánicas y oficinas*.
- [4] Reglamento (UE) 2015/2424, que modifica el Reglamento (CE) 2195/2007 sobre el Vocabulario Común de Contratos Públicos (CPV).
- [5] Polars. *User guide*. <https://pola.rs>
- [6] Apache Software Foundation. *Apache Spark documentation*. <https://spark.apache.org/docs/latest/>
- [7] DuckDB Foundation. *DuckDB documentation*. <https://duckdb.org/docs/stable/>
- [8] Salesforce. *Tableau documentation*. <https://help.tableau.com/>
- [9] Campoy Rojo, T. (2026). *tfm-licitaciones: pipeline DataOps reproducible para contratación pública*. Repositorio de código en GitHub. <https://github.com/tomcrojo/tfm-licitaciones>. Consulta: 17 de septiembre de 2026.
- [10] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.